



Python for Fixed Income

COURSE 2 · BOND MATH

Clean and Dirty Price

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The price between coupons

Buy 1,000,000 of face of Quorvane at 5.40 percent, settling 30 November 2026

What price is quoted, and how much cash leaves the account?

Two things change from week 1

1. The next coupon is four months away: part of a period
2. Two months of interest belong to the seller: accrued

Today builds on day 7's accrued interest: clean price plus accrued is what the buyer pays.

Today

- 8.1 The quote and the invoice
- 8.2 Part of a period, $w = \text{DSC} / E$
- 8.3 price and dirty_price
- 8.4 On a coupon date: week 1
- 8.5 The final coupon period
- 8.6 yield_to_maturity
- 8.7 The September blotter

The quote and the invoice

99.361551

clean price per 100: the quote, Excel's PRICE

0.875000

accrued interest per 100: day 7

100.236551

dirty price per 100: what the buyer pays

On 1,000,000 of face	Dollars
Principal: face x clean / 100	993,615.51
Accrued: face x accrued / 100	8,750.00
Invoice: principal + accrued	1,002,365.51

Excel side by side (tested in Excel)

<code>=PRICE(s,m,5.25/100,5.4/100,100,2,0)</code>	99.361551
<code>=PRICE(...)+100*5.25/100/2*COUPDAYBS(...)/COUPDAYS(...)</code>	100.236551

s = DATE(2026,11,30), m = DATE(2031,9,30); the same s, m, 2, 0 in COUPDAYBS and COUPDAYS

The ticket shows clean. Settlement moves the dirty amount.

Part of a period: $w = DSC / E$

$$w = DSC / E = 120 / 180 = 0.6667$$

k	Coupon date	Periods	Cash flow	PV
0	2027-03-31	0.667	2.625	2.578788
1	2027-09-30	1.667	2.625	2.510991
...	...	...	...	...
9	2031-09-30	9.667	102.625	79.324172

Sum of the PVs: 100.236551, the dirty price. Less accrued: 99.361551, PRICE.

Discount first, subtract accrued second.

Excel side by side (tested in Excel)

B1: =COUPDAYBS(s,m,2,0)	60
B2: =COUPDAYS(s,m,2,0)	180
B3: =COUPNUM(s,m,2,0)	10
B4: =(B2-B1)/B2	w
B6 down: =IF(A6=\$B\$3-1,...)/(1+y/2)^(\$B\$4+A6)	PV
B16: =SUM(B6:B15)	100.236551

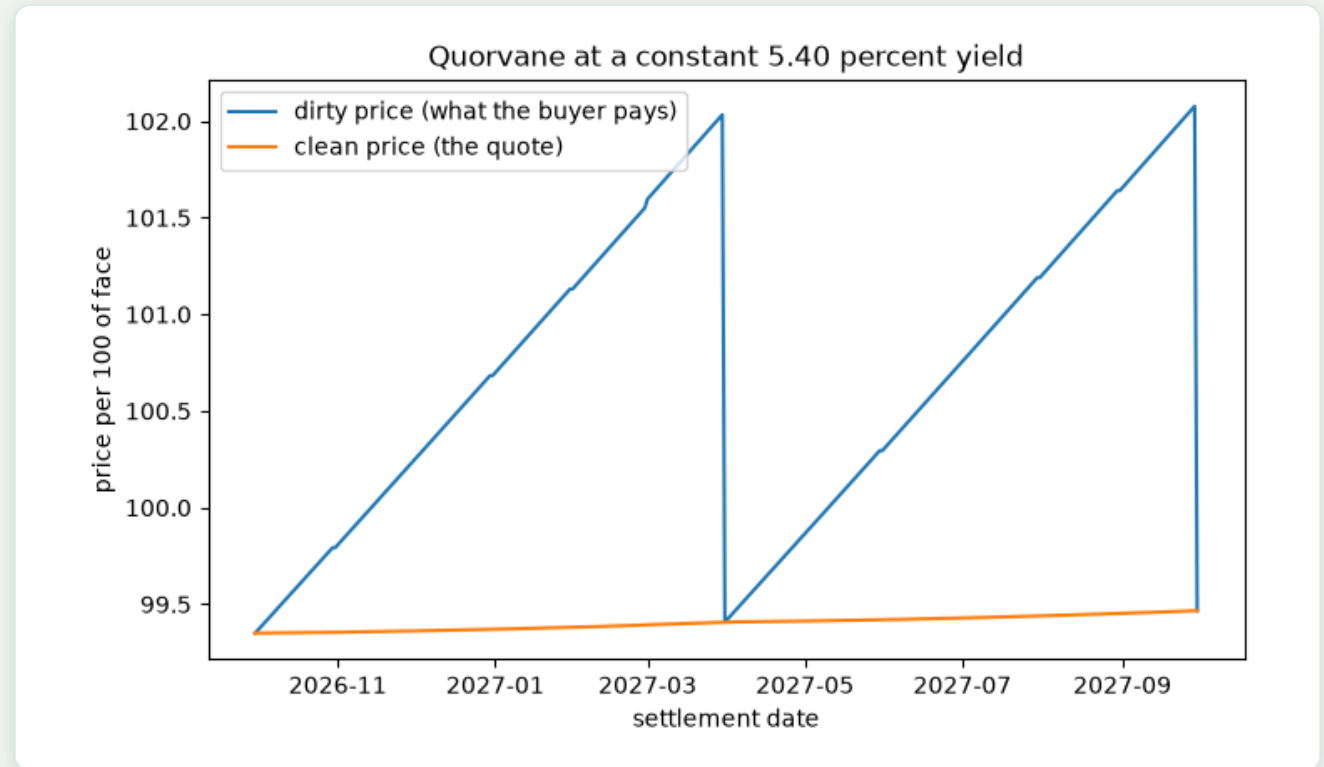
One cell: =-PV(y/2,COUPNUM(...),c/2,100)*(1+y/2)^(COUPDAYBS/COUPDAYS)
gives the same dirty price.

$$s = \text{DATE}(2026,11,30), m = \text{DATE}(2031,9,30), y = 5.4/100, c = 5.25$$

price and dirty_price, and why quotes are clean

```
bondmath.price(settlement, maturity,  
               coupon_pct, yield_pct)  
# clean price per 100, as PRICE  
bondmath.dirty_price(...)  
# price + accrued_interest
```

Bond	Clean	Dirty
Quorvane Energy	99.3616	100.2366
Dunmarrow Rail	99.9898	101.0106
Pellwick Paper	100.3882	101.1799
Halvering Foods	98.6225	99.8725



Dirty climbs, then drops a coupon. Clean moves smoothly: a change in the quote is a change in value.

On a coupon date it equals week 1

A = 0

nothing accrued on 30
September 2026

w = 1

the next coupon one whole
period away

The new function contains the old one as a special case. The old answers must not move.

Bond	price, 30 Sep	Excel -PV, week 1
Quorvane Energy	99.350327	99.350327
Dunmarrow Rail	100.000000	100.000000
Pellwick Paper	100.415887	100.415887
Halvering Foods	98.622736	98.622736

price equals price_from_yield to the twelfth decimal; the test checks every reference row that settles on a coupon date.

The final coupon period: simple interest

One coupon left: Excel's PRICE

$$\text{dirty} = (100 + c/f) / (1 + w \times y/f)$$

Every other period compounds

$$(1 + y/f) ** (w + k)$$

Row edge_final_period: 5 percent coupon, 6 percent yield, settled 30 September 2026, maturing 15 February 2027, $w = 0.75$.

Neither is wrong. The convention decides, and bondmath follows Excel.

99.619499

simple interest: Excel's PRICE and
bondmath.price

99.627670

compound, for comparison

Gap: 0.008171 per 100, 81.71 dollars on 1,000,000 of face. Tested on all final-period rows of the reference file.

yield_to_maturity, as Excel's YIELD

Bond	Quote	Yield 30 Sep	Yield 30 Nov
Quorvane Energy	99.125	5.4523	5.4565
Dunmarrow Rail	100.500	6.0364	6.0328
Pellwick Paper	100.875	4.4353	4.4159
Halvering Foods	97.750	7.8286	7.8300

Same clean price, two months on: every yield moves away from its coupon. A description of the formula, not a forecast.

Final period near a month end

6.5 percent coupon, settled 15 December 2026, maturing 28 February 2027.

Step	Result
PRICE at 5 percent	100.289734
YIELD of that price	5.1370 percent
price at that yield	100.260880

PRICE uses $E - A = 75$ days. YIELD counts DSR on 30/360: 73 days.

Each does what it documents. bondmath follows Excel in both.

Invoice amounts for the September blotter

374

trades, clean price and principal only

1,809,789.22

dollars of accrued at each settle_date

119,283,538.57

dollars invoiced: principal + accrued

Side	Trades	Principal	Accrued	Invoice
BUY (desk pays)	197	56,556,005.45	990,620.95	57,546,626.40
SELL (desk receives)	177	60,917,743.90	819,168.26	61,736,912.16

Tested in Excel for every trade: $\text{face} \times (\text{price} + \text{accrued formula}) / 100$.

2 trades settled on a coupon date: nothing accrued. Most accrued: T0151, 57,300.35 dollars on 2,500,000 of face.

Accrued is not a cost or a gain. A cash reconciliation on principal alone misses it.

Today's Excel, every formula tested

Job	Excel (tested)	Returns
Clean price	=PRICE (DATE (2026,11,30),DATE (2031,9,30),5.25/100,5.4/100,100,2,0)	99.361551
Accrued	=100*5.25/100/2*COUPDAYBS (...)/COUPDAYS (...)	0.875000
Dirty price	=PRICE (...)+100*5.25/100/2*COUPDAYBS (...)/COUPDAYS (...)	100.236551
Written out	=IF (...)/(1+5.4/100/2)^(\$B\$4+A6) down, then =SUM(B6:B15)	100.236551
One cell	=-PV(5.4/100/2,COUPNUM (...),5.25/2,100)*(1+5.4/100/2)^(A/E)	100.236551
Final period	=PRICE (DATE (2026,9,30),DATE (2027,2,15),5/100,6/100,100,2,0)	99.619499
Yield	=YIELD (DATE (2026,11,30),DATE (2031,9,30),5.25/100,99.125,100,2,0)*100	5.456532

PRICE returns clean and YIELD takes clean. The dirty price you build yourself: add the accrued.

git revert: undo a commit, keep the history

The colleague's commit

```
- dirty = (100 + coupon) / (1 + w * period_yield)
+ dirty = (100 + coupon) / (1 + period_yield) ** w
```

pytest: 3 failed, each on edge_final_period. The last commit is the broken file, so restore cannot help.

git revert --no-edit HEAD, then git log --oneline

```
abc1234 Revert "Discount the final period like every other period"
abc1234 Discount the final period like every other period
abc1234 Add price, dirty_price, and yield_to_maturity, tested aga...
abc1234 Start of day 8: bondmath.py and test_bondmath.py as day 7...
```

restore undoes a change you have not committed. revert undoes one you have, and keeps the record.

The revert is a new commit that does the opposite of the old one. Never rewrite a history someone may have copied; add to it.

Bring it home: paste, test, commit, revert

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

Two module cells and four test cells at the end of your files, without the %%writefile line.

3

Test

python -m pytest -q: 23 passed, or 30 with the exercises of days 1 to 7.

4

Commit

git diff --stat, git add, git commit with a subject and a body.

5

Revert

Make the colleague's change, commit -am, see 3 fail, git revert --no-edit HEAD.

```
git commit -am "..."
```

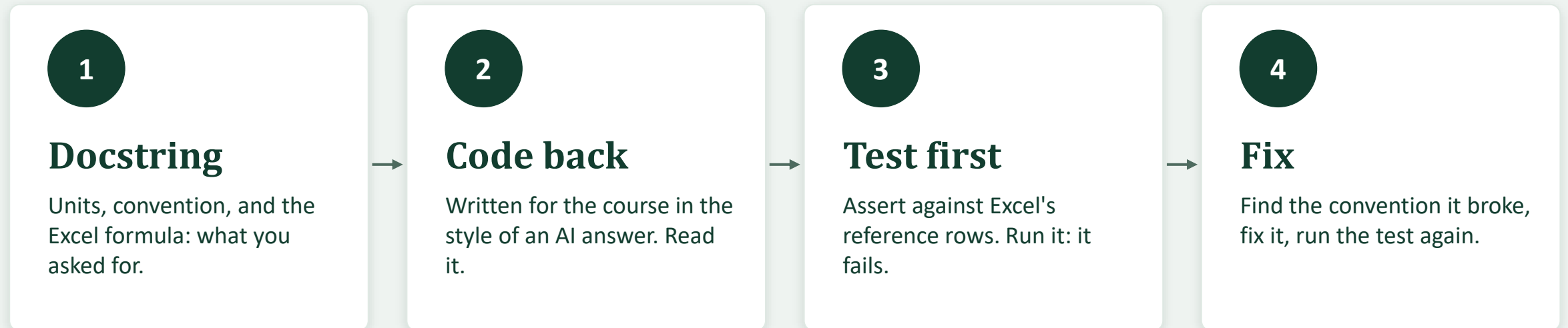
```
python -m pytest -q --tb=no
```

```
git revert --no-edit HEAD
```

In Colab: download bondmath.py and test_bondmath.py from the work folder.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.



Today: price returns the dirty price as the clean price: the accrued is never taken off.

Code that runs is not the same as code that is right.

What you can do now

- Tell the quote (clean) from the invoice (dirty = clean + accrued)
- Discount over part of a period, $w = DSC / E$, in Python and in Excel
- price, dirty_price, yield_to_maturity, matching PRICE and YIELD
- Price the final coupon period with simple interest, as Excel does
- Invoice a trade blotter; undo a commit with git revert

Week 2, Day 4 Duration and DV01: how much the price moves when the yield moves